Universidad Nacional De Colombia

Facultad de minas

Programación Orientada a Objetos
Actividad 6: Valor 20%
Julio 2026

Docente
Walter Hugo Arboleda

Estudiante
Iván Camilo Barragán Echavarría
ibarragan@unal.edu.co

UNIVERSIDAD NACIONAL DE COLOMBIA

Enlace repositorio: https://github.com/AtodeKimeru/hello-java/tree/main/Actividad6/src/main/java

Ejercicio 2.10 Sobrecarga de Métodos Página 127

Para compilar este ejercicio usando Maven, se debe correr en terminal la siguiente línea de código:

“””
mvn compile exec:java '-Dexec.mainClass=Orders.Order'
“””

Asegura estar en el directorio del proyecto Actividad6/

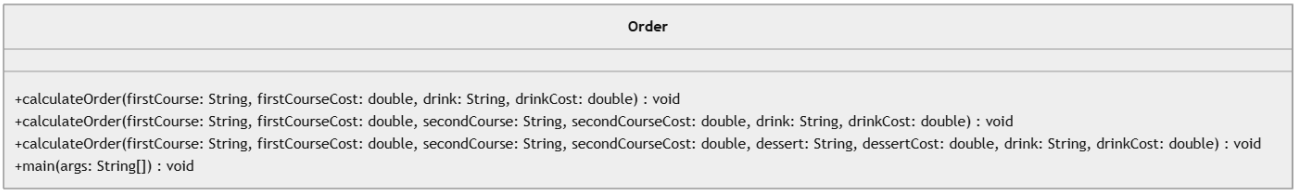
Código fuente

```
package Orders;


public class Order {


   /**
    * Calculates the cost for a 2-item order (First Course + Drink).
    */
   public void calculateOrder(String firstCourse, double firstCourseCost, String drink, double drinkCost) {
       double total = firstCourseCost + drinkCost;
       System.out.println("El costo de " + firstCourse + " y " + drink + " es = $" + total);
   }


   /**
    * Calculates the cost for a 3-item order (First Course + Second Course +
    * Drink).
```

```
    */
    public void calculateOrder(String firstCourse, double firstCourseCost, String secondCourse,
double secondCourseCost,
            String drink, double drinkCost) {
        double total = firstCourseCost + secondCourseCost + drinkCost;
        System.out.println("El costo de " + firstCourse + " + " + secondCourse + " + " + drink + " es =
$" + total);
    }

    /**
     * Calculates the cost for a 4-item order (First Course + Second Course +
     * Dessert + Drink).
     */
    public void calculateOrder(String firstCourse, double firstCourseCost, String secondCourse,
double secondCourseCost,
            String dessert, double dessertCost, String drink, double drinkCost) {
        double total = firstCourseCost + secondCourseCost + dessertCost + drinkCost;
        System.out.println("El costo de " + firstCourse + " + " + secondCourse + " + " + drink + " + "
+ dessert
                + " es = $" + total);
    }

    public static void main(String[] args) {
        Order order1 = new Order();
        order1.calculateOrder("Sancocho", 5000, "Gaseosa", 2000);

        Order order2 = new Order();
        order2.calculateOrder("Crema de verduras", 5000, "Churrasco", 6000, "Gaseosa", 2000);
```

```
    Order order3 = new Order();

    order3.calculateOrder("Crema de espinacas", 5000, "Salmón", 10000, "Tiramisú", 5000,
"Gaseosa", 2000);

  }

}
```

Diagrama De Clases UML



Ejercicio 2.11 Sobrecarga de Constructores Página 132

Para compilar este ejercicio usando Maven, se debe correr en terminal la siguiente línea de código:

“””
mvn compile exec:java '-Dexec.mainClass=ScientificArticle'
“””

Asegura estar en el directorio del proyecto Actividad6/

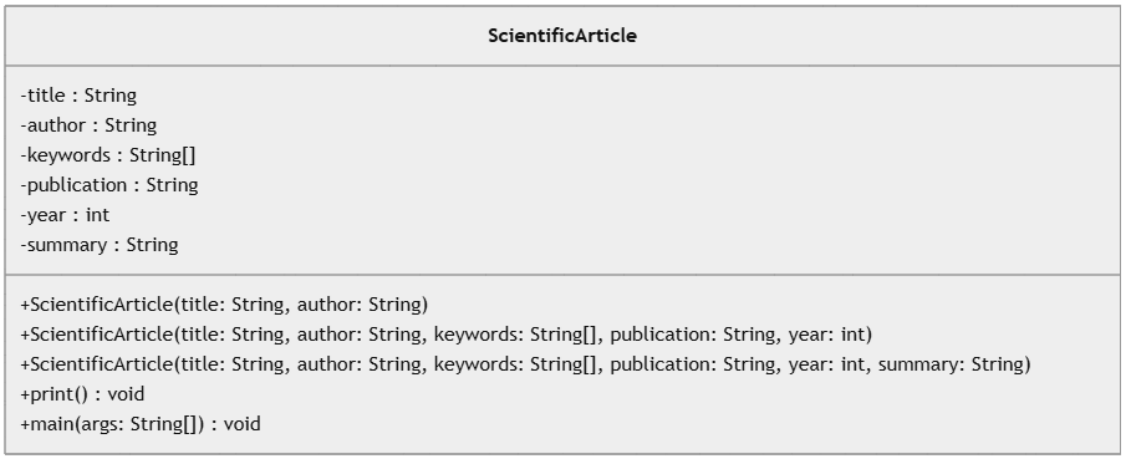
Código fuente

```
import java.util.Arrays;


public class ScientificArticle {

  private String title;

  private String author;

  private String[] keywords = new String[3];
```

```
   private String publication;
   private int year;
   private String summary;

   public ScientificArticle(String title, String author) {
      this.title = title;
      this.author = author;
   }

   public ScientificArticle(String title, String author, String[] keywords, String publication, int
year) {
      this(title, author);
      this.keywords = keywords;
      this.publication = publication;
      this.year = year;
   }

   public ScientificArticle(String title, String author, String[] keywords, String publication, int
year,
         String summary) {
      this(title, author, keywords, publication, year);
      this.summary = summary;
   }

   public void print() {
      System.out.println("Título del artículo = " + title);
      System.out.println("Autor del artículo = " + author);
      System.out.println("Palabras clave = ");
```

```
        if (keywords != null) {
            for (String keyword : keywords) {
                System.out.println(keyword);
            }
        }
        System.out.println("Publicación = " + publication);
        System.out.println("Año = " + year);
        System.out.println("Resumen = " + summary);
    }

    public static void main(String[] args) {
        String[] keywords = { "Física", "Espacio", "Tiempo" };
        ScientificArticle article = new ScientificArticle(
                "La teoría especial de la relatividad",
                "Albert Einstein",
                keywords,
                "Anales de Física",
                1913,
                "Las leyes de la física son las mismas en todos los sistemas de referencia inerciales.");
        article.print();
    }
}
```

Diagrama De Clases UML



Ejercicio 4.4 Polimorfismo Página 231

Para compilar este ejercicio usando Maven, se debe correr en terminal la siguiente línea de código:

“””
mvn compile exec:java '-Dexec.mainClass=Professors.TestPolymorphism'
“””

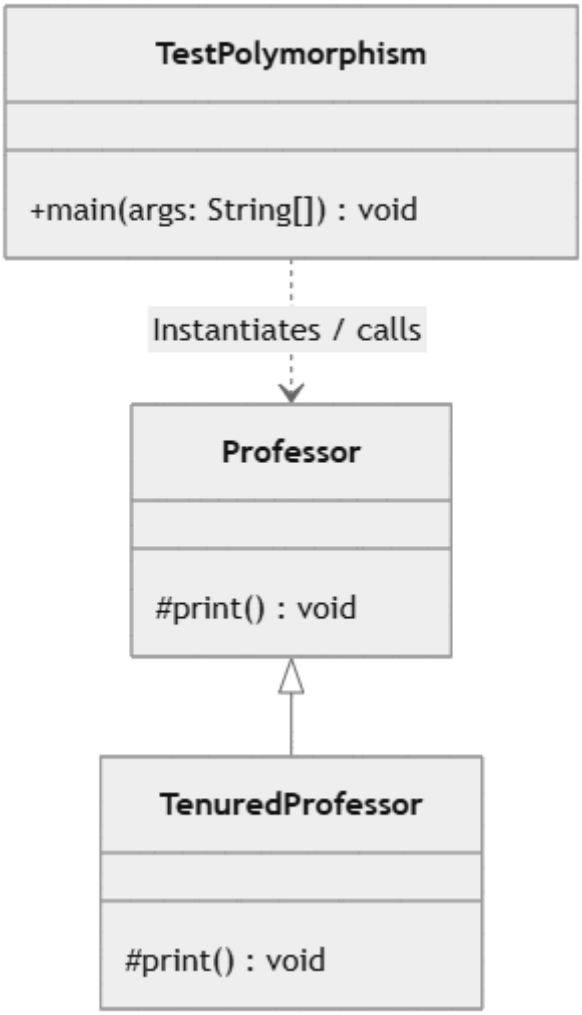
Asegura estar en el directorio del proyecto Actividad6/

Código fuente

```
package Professors;


public class TestPolymorphism  {


   public static void main(String[] args) {

      Professor professor1 = new TenuredProfessor();

      professor1.print();

   }

}


public class Professor {
```

```
    protected void print() {
        System.out.println("Es un profesor.");
    }
}

public class TenuredProfessor extends Professor {

    @Override
    protected void print() {
        System.out.println("Es un profesor titular.");
    }
}
```

Diagrama De Clases UML



Ejercicio 4.6 Métodos Polimórficos Página 239

Para compilar este ejercicio usando Maven, se debe correr en terminal la siguiente línea de código:

“””

mvn compile exec:java "-Dexec.mainClass=Professors.Ejercicio46"

“””

Asegura estar en el directorio del proyecto Actividad6/

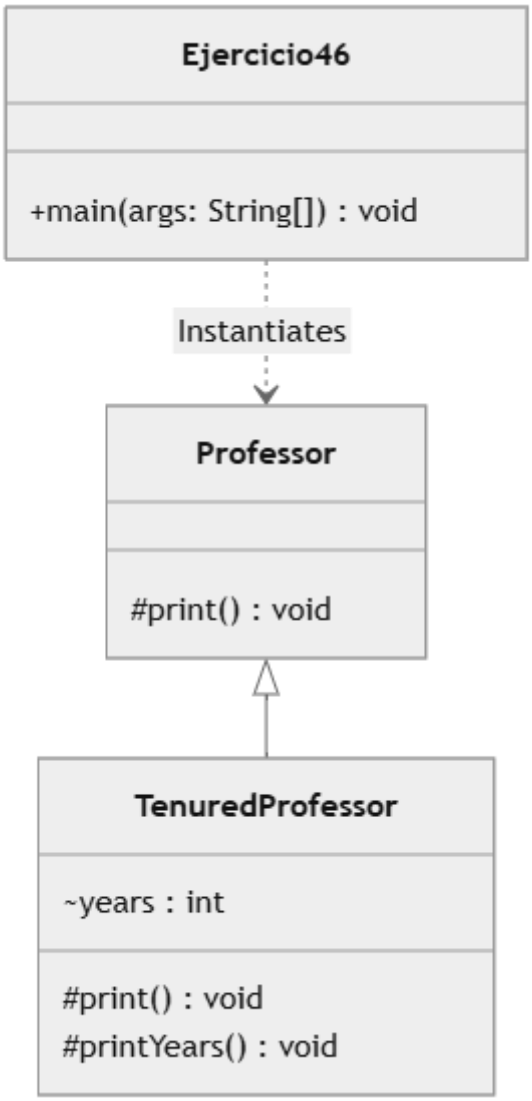
Código fuente

package Professors;

```
public class Ejercicio46 {
    public static void main(String[] args) {
        Professor professor1 = new Professor();
        professor1.print();
        System.out.println("================================");

        TenuredProfessor professor2 = new TenuredProfessor();
        professor2.print();
        professor2.printYears();
    }
}

public class Professor {

    protected void print() {
        System.out.println("Es un profesor.");
    }
}

public class TenuredProfessor extends Professor {

    int years = 0;

    @Override
    protected void print() {
        System.out.println("Es un profesor titular.");
    }
```

```
    protected void printYears() {
        System.out.println("Años = " + years);
    }
}
```

Diagrama De Clases UML



Ejercicio 4.7 Clases Abstractas Página 244

Para compilar este ejercicio usando Maven, se debe correr en terminal la siguiente línea de código:

“””
mvn compile exec:java "-Dexec.mainClass=Animals.TestAnimals"
“””

Asegura estar en el directorio del proyecto Actividad6/

Código fuente

```
package Animals;


public class TestAnimals {


   public static void main(String[] args) {
      Animal[] animals = new Animal[4];
      animals[0] = new Cat();
      animals[1] = new Dog();
      animals[2] = new Wolf();
      animals[3] = new Lion();


      for (Animal animal : animals) {
         System.out.println(animal.getScientificName());
         System.out.println("Sonido: " + animal.getSound());
         System.out.println("Alimentos: " + animal.getFood());
         System.out.println("Hábitat: " + animal.getHabitat());
         System.out.println();
      }
```

```
    }
}

public abstract class Animal {

    protected String sound;
    protected String food;
    protected String habitat;
    protected String scientificName;

    public abstract String getScientificName();

    public abstract String getSound();

    public abstract String getFood();

    public abstract String getHabitat();
}

package Animals;

public abstract class Canid extends Animal {
}

public abstract class Felid extends Animal {
}
```

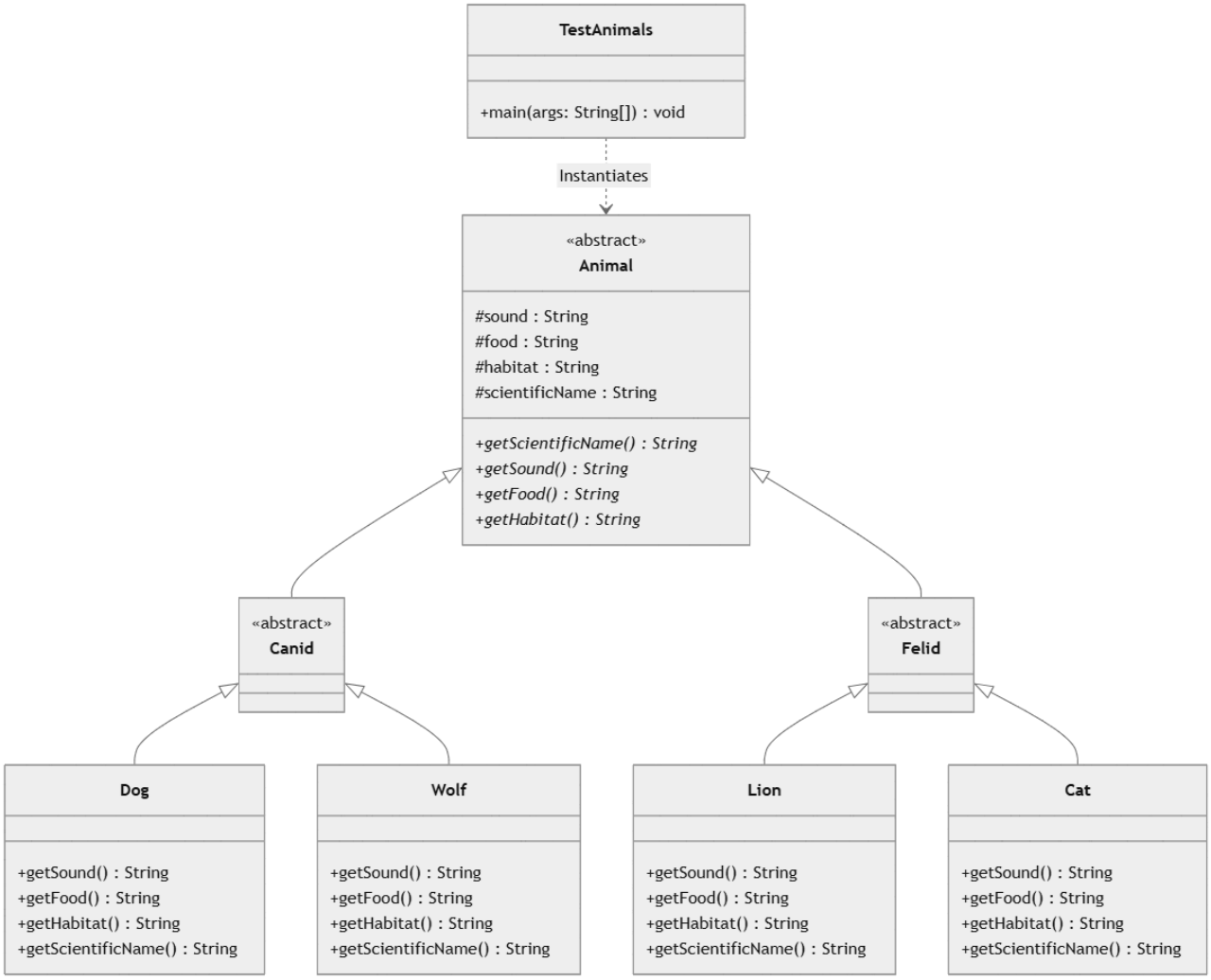
```
public class Dog extends Canid {

    @Override
    public String getSound() {
        return "Ladrido";
    }

    @Override
    public String getFood() {
        return "Carnívoro";
    }

    @Override
    public String getHabitat() {
        return "Doméstico";
    }

    @Override
    public String getScientificName() {
        return "Canis lupus familiaris";
    }
}

public class Wolf extends Canid {

    @Override
```

```
    public String getSound() {
        return "Aullido";
    }

    @Override
    public String getFood() {
        return "Carnívoro";
    }

    @Override
    public String getHabitat() {
        return "Bosque";
    }

    @Override
    public String getScientificName() {
        return "Canis lupus";
    }
}

public class Cat extends Felid {

    @Override
    public String getSound() {
        return "Maullido";
    }
```

```
    @Override
    public String getFood() {
        return "Ratones";
    }

    @Override
    public String getHabitat() {
        return "Doméstico";
    }

    @Override
    public String getScientificName() {
        return "Felis silvestris catus";
    }
}

public class Lion extends Felid {

    @Override
    public String getSound() {
        return "Rugido";
    }

    @Override
    public String getFood() {
        return "Carnívoro";
    }
```

```
    @Override
    public String getHabitat() {
        return "Praderas";
    }

    @Override
    public String getScientificName() {
        return "Panthera leo";
    }
}
```

Diagrama De Clases UML


TestAnimals
+main(args: String[]) : void
Instantiates
«abstract»
Animal
#sound : String
#food : String
#habitat : String
#scientificName : String
+getScientificName() : String
+getSound() : String
+getFood() : String
+getHabitat() : String
«abstract»
Canid
«abstract»
Felid
Dog
+getSound() : String
+getFood() : String
+getHabitat() : String
+getScientificName() : String
Wolf
+getSound() : String
+getFood() : String
+getHabitat() : String
+getScientificName() : String
Lion
+getSound() : String
+getFood() : String
+getHabitat() : String
+getScientificName() : String
Cat
+getSound() : String
+getFood() : String
+getHabitat() : String
+getScientificName() : String